

Post-Training a Vision-Language Model on Magic Tricks: A Research Briefing

Prepared for: meeting prep, BASIS Peoria senior researcher **Purpose:** decode a rough set of meeting notes into a technically credible research plan, teach the underlying ML concepts from first principles, and identify a first experiment **Method:** every non-obvious claim below is checked against a primary source (arXiv paper, official docs, or GitHub repo) found via live web search on September 1, 2026. Where I could not verify something, I say so directly instead of guessing.

How to read this document

This is long because the assignment is broad and the request asked for depth. You do not need to read it top to bottom before your meeting. Part 15 and the 7-Day Plan at the end are the two sections that matter most operationally. Parts 1-14 are the reference material that explains *why* the plan looks the way it does.

I am a language model without access to your meeting, your professor, or their prior publications. Everything below about "what the professor probably meant" is inference from your notes plus the shape of the current literature, not a confirmed fact. I flag confidence levels throughout, and Part 17 (the uncertainty ledger) separates what is actually known from what is guessed.

Uncertainty Ledger (short form - full version at the end)

What we know for certain:

- Your notes contain real, correctly-spelled-adjacent technical terms: GRPO, PPO, RLHF (garbled as "rhelf"), Bradley-Terry model. These are all real, currently active concepts in ML post-training, not something transcribed incorrectly into nonsense.
- GRPO and PPO are reinforcement learning algorithms used to post-train language and vision-language models. Bradley-Terry is a statistical model used inside reward modeling. All three routinely appear together in the same papers.
- There is no well-established, named subfield of "AI does magic tricks." The nearest real precedent (Zaghi-Lara et al., 2019, discussed in Part 7) used pose-tracking, not a vision-language model, and predates the RLHF-for-VLMs literature entirely.

What we infer (medium-to-high confidence):

- The professor is very likely asking you to explore whether GRPO/PPO-style post-training with a learned or rule-based reward, informed by human preference data modeled via Bradley-Terry, can make a vision-language model better at some visual-reasoning task involving magic tricks (explaining tricks, detecting the secret mechanism, following the procedure, etc.).
- "Magic tricks" is probably being used as a *stand-in domain* for a harder underlying capability: visual reasoning under occlusion, misdirection, or hidden state, rather than magic being the actual object of study. This is a common pattern in ML research (pick a vivid, tractable domain to probe a general capability).

What we do not know (must ask the professor):

- Which specific interpretation from Part 6 (A through L) they mean.
- Whether they want image-only or video input.
- Whether they already have a dataset, compute budget, or specific VLM in mind.
- Whether "mentalism" is meant literally (predicting/reading minds

as a task format) or loosely (as another word for "tricks that seem to defy normal reasoning").

PART 1 - Decoding your notes, phrase by phrase

Your raw notes: *"magic tricks post training in current vision model to do magic tricks, grpo, mentalism tricks, post train model to do magic tricks, visual learning model, grpo ppo design some reward function, human feedback, rhelf, bradley tetty model."*

I am not silently correcting these - here is the term-by-term reasoning.

What you wrote	Most likely correct term	Why	Role in the project
"post training"	Post-training	Standard, correctly spelled term. Refers to any training stage applied <i>after</i> a model's initial large-scale pretraining - instruction tuning, RLHF, DPO, GRPO, etc.	This is the overall category of technique the professor wants you to apply.

"current vision model"	Vision-language model (VLM) , current generation (e.g., Qwen-VL, LLaVA, InternVL family)	"Vision model" alone usually means an image classifier (like a plain CNN) with no language output. Since the task involves reasoning/ explaining, a VLM (image+text in, text out) is almost certainly meant.	This is the base model you post-train.
"magic tricks"	Magic tricks (literal)	No garbling here. This is genuinely the stated domain.	The task domain: video/image content of a magic trick performance.
"mentalism tricks"	Mentalism (literal)	A real magic subgenre: tricks that appear to demonstrate mind-reading, prediction, or psychological influence, as opposed to sleight-of-hand with physical objects.	Possibly a narrower sub-case the professor wants, or just an example given in conversation.

"grpo"	GRPO - Group Relative Policy Optimization	Correctly transcribed as-is; it is a real, specific algorithm name, not a mishearing of something else. Introduced by DeepSeek in the DeepSeekMath paper (Shao et al., 2024, arXiv:2402.03300) and popularized by DeepSeek-R1 (2025).	The likely post-training algorithm. See Part 4.
"ppo"	PPO - Proximal Policy Optimization	Correctly transcribed. Introduced by Schulman et al., 2017 (arXiv:1707.06347); it is the RL algorithm underlying the original InstructGPT-style RLHF pipeline.	Alternative or predecessor algorithm to GRPO. See Part 4.
"design some reward function"	Reward function design	Literal, standard ML terminology for the function that scores model outputs during RL.	The other half of the RL loop besides the algorithm - see Part 9.
"human feedback"	Human feedback (literal), used inside RLHF	Literal.	The data source used to train a reward or preference model. See Part 8.

"rhellf"

**RLHF -
Reinforcement
Learning from
Human
Feedback**

"Rhellf" is a very close phonetic match to someone saying the acronym "R-L-H-F" quickly, or to the informal pronunciation "relf"/"rhelf" some people use. I am fairly confident this is RLHF, not RLAIF - see the next row for why I did not choose that instead.

The overall alignment paradigm: fit a reward/preference model to human judgments, then optimize the policy against it (via PPO, GRPO, DPO, etc.). Introduced at scale for LLMs in Ouyang et al., 2022 ("InstructGPT," arXiv:2203.02155).

("rhellf" alternate reading)	RLAIF - Reinforcement Learning from AI Feedback	You explicitly asked me to consider this. RLAIF (Bai et al., 2022, "Constitutional AI," arXiv:2212.08073; Lee et al., 2023, arXiv:2309.00267) replaces the human labeler with another AI model's judgments. Phonetically "rhellf" fits RLHF better than RLAIF (which would more likely garble to something like "relayf" or "arlayf"). I am flagging this as a real possibility only because you listed "human feedback" <i>and</i> "rhellf" as two separate bullet points in your notes - if they were meant to be the same term, that's redundant, which makes me lean slightly toward "rhellf" = RLHF stated a second time for emphasis, or genuinely = RLAIF as the AI-feedback alternative your	If RLAIF: an AI judge (e.g., a larger VLM or the model itself) generates preference labels instead of/ alongside human ones, which matters enormously for how much manual annotation your dataset build will need.
------------------------------------	--	---	---

		professor mentioned as an option. Ask the professor to disambiguate this.	
"bradley tetty model"	Bradley-Terry model	"Tetty" is a clean phonetic near-miss for "Terry." This is a specific, real, 1952 statistical model (Bradley, R.A. & Terry, M.E., "Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons," <i>Biometrika</i> 39(3/4):324-345) that underlies almost every modern reward model trained on pairwise human preferences.	The statistical backbone for turning "human A preferred output 1 over output 2" into a trainable numeric reward. See Part 5.
"visual learning model"	Likely a loose restatement of vision-language model or visual reasoning	Probably not a distinct technical term; more likely your own paraphrase of "a model that learns from visual input."	Same referent as row 2.

Summary interpretation with confidence level: The professor most likely described a project where you take an existing open vision-language model and post-train it (via GRPO and/or PPO) using a reward signal derived from human preference judgments (formalized with a Bradley-Terry preference/reward model, in the RLHF tradition, possibly with an RLAIIF variant), targeting improved performance on some visual-

reasoning task involving magic tricks or mentalism. **Confidence: medium-high on the mechanism (GRPO/PPO + Bradley-Terry + RLHF), low-medium on the exact task definition** (that's Part 6, and it is the biggest open question for your next meeting).

PART 2 - Does this research problem already exist in the literature?

I searched specifically for: AI understanding/performing magic tricks, visual reasoning about tricks, misdirection detection, procedural video learning, and RL post-training of VLMs with preference data. Findings, split by how directly relevant they are:

Directly on point: AI + magic tricks (rare, and not what you'd build)

- **Zaghi-Lara, R., Gea, M.A., Camí, J., Martínez, L.M., Gómez-Marín, A. (2019). "Playing magic tricks to deep neural networks untangles human deception." arXiv:1908.07446.** This is the closest thing to prior work on "AI and magic tricks" that I could find. It used DeepLabCut (a pose-estimation tool, not a VLM or LLM) to track a magician's fingers and coin positions frame-by-frame during real sleight-of-hand tricks, then compared which tricks fooled the tracking algorithm versus which fooled human observers. It is a neuroscience/cognitive-science paper published to study *human* perception, using AI as an instrument, not a paper about building a better trick-understanding AI. **Relevance:** strong precedent that "magic + AI" is a legitimate, published research angle, but it used 2018-era pose tracking, not language-model post-training. It does not touch GRPO, PPO, RLHF, or VLMs at all, because those tools were not yet dominant in 2019. This is good news for novelty (see Part 14): nobody has done the VLM/RLHF version of this.
- **Manufacturing Magic and Computational Creativity** (Frontiers in Psychology, Kuhn-adjacent literature, PMC4901183). This

paper explicitly states, as of its writing, that "*an as yet unexplored prospect is a workable computational model of misdirection*" - i.e., a leading voice in the psychology-of-magic field was, at the time, flagging this exact gap as open. **Relevance:** supports that a computational/AI model of misdirection is recognized by domain experts as unclaimed territory, strengthening the novelty case for a well-scoped project.

The cognitive-science-of-magic literature (foundational, not AI, but likely the theoretical grounding your professor draws on)

A large body of work by Gustav Kuhn and collaborators formally taxonomizes how magic tricks fool people:

- Kuhn, G., Caffaratti, H.A., Teszka, R., Rensink, R.A. (2014). "A psychologically-based taxonomy of misdirection." *Frontiers in Psychology* 5:1392.
- Kuhn, G., Martinez, L.M. (2011). "Misdirection - past, present, and the future." *Frontiers in Human Neuroscience* 5:172.
- Barnhart, A.S., Goldinger, S.D. (2014). "Blinded by magic: eye-movements reveal the misdirection of attention." *Frontiers in Psychology*. Notably, Goldinger is in the Department of Psychological Sciences at **Arizona State University** - if your professor's research connects to this cognitive-science-of-misdirection tradition, this specific lab/line of work is a plausible intellectual neighbor, though I want to be clear I have **no direct evidence** your professor is affiliated with it. **Ask about this connection at your next meeting** rather than assuming it.

Why this matters: this literature gives you a ready-made taxonomy of *what makes a trick work* (attention misdirection, memory misdirection, reasoning misdirection - three classes per Kuhn et al. 2014). That taxonomy is directly usable as a label schema for a dataset (Part 7) or as categories for a reward function's rubric (Part 9), even though none of it involves training a model.

Adjacent and highly relevant: VLM post-training with

RLHF/RL, not about magic but about the exact mechanism

- **Sun, Z. et al. (2023). "Aligning Large Multimodal Models with Factually Augmented RLHF" (LLaVA-RLHF).** [arXiv:2309.14525](#). Trains a VLM with a human-preference reward model (Bradley-Terry-based) plus PPO, specifically to reduce hallucination. This is essentially the exact recipe implied by your notes, just applied to a different target behavior (factual grounding instead of magic-trick understanding).
- **Yu, T. et al. (2023). "RLHF-V: Towards Trustworthy MLLMs via Behavior Alignment from Fine-grained Correctional Human Feedback."** [arXiv:2312.00849](#), CVPR 2024. Collects fine-grained human *corrections* (not just A-vs-B preferences) on VLM outputs and trains with a dense variant of DPO. Useful as an alternative to plain pairwise preferences (see Part 8).
- **Yu, T. et al. (2024). "RLAIF-V: Aligning MLLMs through Open-Source AI Feedback for Super GPT-4V Trustworthiness."** [arXiv:2405.17220](#). The AI-feedback (no human labeler) version of the above - directly relevant if "rhellf" in your notes turns out to mean RLAIF.
- **VLM-R1 team (2025). "VLM-R1: A Stable and Generalizable R1-style Large Vision-Language Model."** [arXiv:2504.07615](#). Open-source framework (built on Open-R1) that applies GRPO specifically to vision-language models for tasks like object detection and grounding, with released code at github.com/om-ai-lab/VLM-R1. This is close to a template you could adapt.
- **Wang, Y. et al. (2024). "RL-VLM-F: Reinforcement Learning from Vision Language Foundation Model Feedback."** [arXiv:2402.03681](#). Uses a VLM itself as the source of reward signal for training a *separate* policy - relevant if your project ends up using an AI judge rather than a from-scratch human-labeled reward model.
- **Rocamonde, J. et al. (2023). "Vision-Language Models are Zero-Shot Reward Models for Reinforcement Learning."** [arXiv:2310.12921](#), ICLR 2024. Uses CLIP-based VLMs directly as reward functions with no fine-tuning, by scoring how well an observation matches a text description. Relevant as a cheap, zero-annotation baseline reward for Part 9.

Adjacent: procedural / instructional video understanding (a good proxy domain if magic-specific video is scarce)

- **Tang, Y. et al. (2019). COIN dataset.** 11,827 instructional videos across 180 tasks, with step-level annotations.
- **Zhukov, D. et al. (2019). CrossTask dataset.** 2,750 videos across 18 primary tasks (and additional related tasks), also step-annotated.
- **Alayrac, J.-C. et al. (2016). NIV (Narrated Instructional Videos).** 150 videos, 5 daily tasks. These are the standard benchmarks for "procedure planning in instructional videos" (predicting the sequence of steps between a start and goal state) - structurally similar to what you'd need for "predict the sequence of a magic trick," except none of them contain magic content. I found derivative work using CrossTask clips reshuffled to test causal/temporal reasoning in VLMs (referenced as "VCRBench" in a 2025 paper on cross-task learning) - I could not fully verify the authorship of that specific benchmark from search results alone, so I am flagging it as **unconfirmed** rather than citing it as fact. It is worth asking the professor if they know it.

Adjacent: does an AI-vs-magic *benchmark* (not training method) already exist?

I found **no evidence** of a published benchmark specifically titled around "magic trick understanding" for LLMs/VLMs (unlike, say, IllusionBench for optical illusions, or HandVQA for hand-pose reasoning, both of which do exist and are cited below). This absence is itself informative for the novelty question in Part 14: if the professor wants a benchmark/dataset contribution, that piece looks genuinely unclaimed. I cannot rule out that something exists that simply didn't surface in my searches - if you find one, it changes the novelty calculus significantly.

Bottom line on Part 2

There is **no existing "AI does magic tricks" subfield** in the ML sense

your notes imply. There **is** a mature body of RLHF-for-VLM methodology (LLaVA-RLHF, RLHF-V, RLAIF-V, VLM-R1) that is domain-agnostic and directly reusable, and there **is** a mature cognitive-science-of-magic literature that supplies theory and task structure but no ML methods. Your project sits at the untouched intersection of these two mature fields. That is a coherent, fundable-sounding research niche, not a red flag - but it also means there is no existing paper to simply reproduce, which raises the importance of a very well-scoped first prototype (Part 12, Part 15).

PART 3 - The likely technical pipeline

Your prompt suggested this candidate pipeline:

1. Start with an existing VLM.
2. Build a dataset of magic-trick demonstrations.
3. Give the model videos/images/questions/prompts.
4. Collect human demonstrations, labels, rankings, critiques, or preferences.
5. Train a reward or preference model.
6. Use a post-training algorithm (PPO, GRPO, DPO, ...).
7. Optimize the VLM against the reward signal.
8. Evaluate whether the model actually improved.

Having checked this against LLaVA-RLHF, RLHF-V, and VLM-R1, **this pipeline is structurally correct** and matches how real VLM-RLHF papers are organized. The one place I would push back is step 5-6: whether you need a *separate trained reward model* at all depends heavily on which algorithm you pick (see Part 4). If you use DPO or GRPO with rule-based/verifiable rewards, you can skip training a standalone reward model entirely. If you use classic PPO-style RLHF (as in LLaVA-RLHF), you do need one.

Revised, algorithm-agnostic pipeline with concrete inputs/outputs at each stage:

Stage	Consumes	Produces	Concrete example
1. Base model selection	Nothing (a choice)	A frozen or LoRA-ready pretrained VLM	Qwen2.5-VL-3B-Instruct downloaded from Hugging Face
2. Task/dataset definition	Chosen interpretation from Part 6	A schema: input format, output format, label type	Input: a 15-second video clip of a coin trick. Output: free-text explanation of the likely mechanism.
3. Raw data collection	Public trick videos or new recordings, task schema	A set of (input,) pairs with no labels yet	50 YouTube close-up magic clips, trimmed and format-standardized
4. Candidate generation	Base VLM + data from step 3	Multiple candidate outputs per input (e.g., 2-8 samples per prompt at temperature > 0)	For each clip, 4 different model-generated explanations
5. Feedback collection	Candidates from step 4	Preference judgments, ratings, or corrections, tied to a rubric	A human (you, or recruited annotators) picks the better of two explanations, or scores 1-5 on "did it name the real mechanism"

6. Reward/ preference model training (optional, algorithm- dependent)	Feedback data from step 5	A scalar- reward function $r(x, y)$ or an implicit one	A small Bradley- Terry reward head trained on your pairwise judgments (see Part 5)
7. Post- training	Base VLM + reward function (learned or rule-based)	An updated ("post-trained") VLM	GRPO run for N steps, sampling groups of 4-8 completions per prompt
8. Evaluation	Pre- and post- training model, held-out test set	Accuracy/ quality deltas, failure analysis	Compare explanation quality on 20 held-out trick clips, pre vs. post

I did **not** find evidence that any alternative pipeline shape (e.g., skipping human feedback and going pure rule-based RLVR, "reinforcement learning with verifiable rewards," which is how GRPO is actually mostly used today in math/code) is what the professor meant, but it is worth naming as the most likely simplification: **if a trick's outcome is programmatically checkable** (e.g., "did the model correctly identify which of 4 boxes hides the coin," a task with a ground-truth answer), you do not need human preference data at all - you can use a rule-based reward exactly like DeepSeekMath's math-correctness reward, and GRPO by itself becomes sufficient without a Bradley-Terry step. This is a significant simplification worth raising with your professor, because it changes whether you need to collect human feedback at all for a first prototype.

PART 4 - PPO vs. GRPO vs. DPO vs. RLHF vs. RLAI

Level 1 (one-sentence intuitions):

- **RLHF** is the overall strategy: "learn what humans want by watching them compare model outputs, then optimize the model to produce more of what they preferred." It is a category, not a single algorithm.
- **RLAIF** is RLHF with an AI model doing the comparing instead of a human.
- **PPO** is a general-purpose RL algorithm (not specific to language models) that updates a policy carefully, in small steps, so it doesn't destroy what the model already knows.
- **GRPO** is a memory-cheaper variant of PPO that scores each output relative to a *group* of other outputs to the same prompt, instead of using a separately trained value network.
- **DPO** skips reinforcement learning entirely: it re-derives the RLHF math so that a simple supervised classification loss on preference pairs achieves (in theory) the same result as RLHF, with no reward model and no sampling loop.

Level 2 (technical explanation):

RLHF, formally. The canonical three-stage recipe from Ouyang et al. (2022, InstructGPT, arXiv:2203.02155):

1. Supervised fine-tuning (SFT) on demonstration data.
2. Train a reward model $r(x, y)$ on human pairwise preferences, using a Bradley-Terry loss (Part 5).
3. Optimize the SFT model with PPO against $r(x, y)$, with a KL-divergence penalty back to the SFT model so it doesn't drift too far and "reward hack."

PPO, formally. From Schulman et al. (2017, arXiv:1707.06347). PPO maximizes an expected reward objective while clipping the probability ratio between the new and old policy so any single update step cannot move the policy too far:

```
L_PPO(theta) = E[ min( ratio * A, clip(ratio, 1-eps, 1+eps) ) ]  
where ratio = pi_theta(y|x) / pi_theta_old(y|x)
```

A here is the **advantage**: roughly, "how much better was this specific

output than what we'd expect on average for this prompt." In the standard PPO-for-LLMs setup, computing $\bar{A}$ well requires training a second neural network (the **value model** or **critic**) whose entire job is to predict the expected reward for a given partial output, so $\bar{A}$ can be computed as (actual reward) minus (predicted expected reward). This critic is typically the same size as the policy model, so PPO effectively needs to hold two full model copies (plus a frozen reference model for the KL penalty, plus the reward model) in memory during training. That is expensive.

GRPO, formally. Introduced in DeepSeekMath (Shao et al., 2024, arXiv:2402.03300) and used at scale in DeepSeek-V3 and DeepSeek-R1. GRPO's key move: for each prompt, sample a **group** of G outputs from the current policy, score every one with the reward function, and compute each output's advantage relative to the group's own mean and standard deviation, instead of using a learned critic:

$$A_i = (r_i - \text{mean}(r_1 \dots r_G)) / \text{std}(r_1 \dots r_G)$$

Then it plugs this A_i into essentially the same clipped objective as PPO (the DeepSeek-V3 technical report, arXiv:2412.19437, gives the full objective, including a KL penalty term against a reference policy). Because there is no critic network, GRPO removes one full model copy from memory, which is why it became the default choice for post-training reasoning models: it is cheaper and, empirically, at least as effective on tasks with per-prompt-verifiable or per-prompt-scoreable rewards.

Concrete toy GRPO example (as you requested). Suppose the prompt is: "Watch this 5-second clip of a coin vanish. Explain how it was done." The policy samples $G = 4$ candidate explanations at temperature 0.8:

- y_1 : "The coin was palmed in the right hand before the reveal." -> scored by your reward function as **$r_1 = 0.9$** (correctly names the real mechanism)
- y_2 : "The coin teleported using quantum tunneling." -> **$r_2 = 0.0$** (nonsensical, no reward)
- y_3 : "The magician distracted the viewer by looking up and away." -> **$r_3 = 0.5$** (correctly identifies misdirection occurred, but doesn't name the specific physical method)
- y_4 : "The coin was dropped into a sleeve." -> **$r_4 = 0.3$** (plausible-

sounding but the video shows short sleeves, factually wrong for this clip)

Group mean = $(0.9 + 0.0 + 0.5 + 0.3) / 4 = 0.425$. Group standard deviation ≈ 0.324 (population std of these four values).

Advantages: $A1 = (0.9 - 0.425)/0.324 \approx +1.47$, $A2 = (0.0 - 0.425)/0.324 \approx -1.31$, $A3 = (0.5 - 0.425)/0.324 \approx +0.23$, $A4 = (0.3 - 0.425)/0.324 \approx -0.39$.

The policy is then pushed to increase the probability of $y1$ substantially (large positive advantage), decrease $y2$'s probability substantially (large negative advantage), and nudge $y3$ up slightly / $y4$ down slightly. Notice the reward function itself did all the "hard" judging here (deciding 0.9 vs 0.5 vs 0.3 vs 0.0) - GRPO's contribution is purely about turning those four raw scores into a stable, low-variance training signal without needing a separate critic network to tell you what an "average" score should have been. **This is exactly why reward function design (Part 9) matters more than which RL algorithm you pick:** GRPO is only as good as the numbers you feed it.

DPO, formally. Rafailov et al. (2023), "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," arXiv:2305.18290, NeurIPS 2023. The key mathematical trick: under a Bradley-Terry preference model, the optimal RLHF-trained policy has a reward function that can be written exactly in terms of the policy's own log-probabilities relative to a frozen reference policy:

$$r_{\text{hat}}(x, y) = \beta * \log(\pi_{\theta}(y|x) / \pi_{\text{ref}}(y|x))$$

Substituting this "implicit reward" back into the Bradley-Terry preference loss gives a loss you can compute directly from a static dataset of (prompt, preferred response, rejected response) triples, with no sampling, no reward model, and no RL loop at all:

$$L_{\text{DPO}}(\theta) = -E[\log(\text{sigmoid}(\beta * \log(\pi_{\theta}(y_w|x) / \pi_{\theta}(y_l|x)))$$

Here y_w is the preferred ("winning") response and y_l is the rejected ("losing") one. DPO is, functionally, ordinary supervised fine-tuning with an unusual loss function; it never needs the model to generate anything

during training.

RLAIF, formally. Bai et al. (2022, arXiv:2212.08073) and Lee et al. (2023, arXiv:2309.00267). Structurally identical to RLHF (still fits a Bradley-Terry-style preference model, still can use PPO or DPO downstream), with the human preference labels replaced by an AI model's judgments, optionally guided by a written "constitution" of principles the judge model is instructed to apply.

Level 3 (application to your project):

Method	Needs human labels?	Needs a trained reward model?	Updates policy directly?	Compute cost	Stability
PPO (classic RLHF)	Yes (for the reward model)	Yes	Via RL rollouts	High (policy + critic + reward model + ref model in memory)	Can be finicky to tune (clip range, K, coefficient, value loss weighting)
GRPO	Optional (labels can come from a rule, an AI judge, or humans - it only needs a reward function)	Optional (can use rule-based or programmatic rewards with no reward model at all)	Via RL rollouts, sampling groups per prompt	Medium (no critic, but still needs multiple samples per prompt at each step, and ideally a fast inference backend like vLLM)	Generally more stable than PPO on reasoning style tasks that's why DeepSeek adopted it

DPO	Yes, in the form of static preference pairs	No	Directly, via supervised-style loss	Low - no sampling loop at all, closer to normal fine-tuning cost	Very straightforward to implement correctly
-----	---	----	-------------------------------------	--	---

RLHF (as an umbrella term)	Yes	Depends on which downstream algorithm you pick	Depends	Depends	Depends
RLAIF	No (AI judge instead)	Same as whichever downstream algorithm	Depends	Cuts annotation cost/time significantly, but introduces AI-judge bias as a new risk	Same as whichever downstream algorithm, plus judge consistency concerns

Which is most plausible for your project, and why: Given that your notes explicitly name **both** GRPO and PPO, and separately name reward-function design, I think the most likely reading is that the professor mentioned GRPO as the modern/preferred choice and PPO as the classical reference point you should understand for context (this is extremely common - people explain GRPO by contrasting it with PPO, exactly like I did above). **My recommendation for your first prototype: start with DPO or rule-based GRPO, not PPO.** PPO's extra

infrastructure (a separate critic network, careful hyperparameter tuning) is not a good use of a first two weeks. Full GRPO with a *learned* reward model is your natural second step. Full PPO-style RLHF, if you ever need it, should be your last step, once the pipeline already works end-to-end with something simpler.

PART 5 - The Bradley-Terry model

Level 1: it's a simple mathematical rule for turning "A beat B" votes into a numeric "skill score" for A and B, originally invented for ranking things like sports teams or wine-tasting preferences, and now repurposed to turn "response A was preferred to response B" into a trainable reward number.

Level 2 (formal): Bradley & Terry (1952) proposed that each item i has a latent "strength" value $p_i > 0$, and the probability that a judge prefers item i over item j is:

$$P(i > j) = p_i / (p_i + p_j)$$

In modern reward modeling it is standard to reparameterize strengths as $p_i = \exp(r_i)$, where r_i is an unbounded real-valued score (this makes r_i easy to output directly from a neural network's final layer).

Substituting:

$$P(i > j) = \exp(r_i) / (\exp(r_i) + \exp(r_j)) = \text{sigmoid}(r_i - r_j)$$

So the model reduces to: **the probability that i beats j is a sigmoid of the difference between their reward scores.** To train a reward model r_θ on a dataset of human preference pairs $D = \{(x, y_w, y_l)\}$ (prompt x , winning response y_w , losing response y_l), you maximize the log-likelihood of the observed preferences, which is equivalent to minimizing:

$$L(\theta) = -E[\log(\text{sigmoid}(r_\theta(x, y_w) - r_\theta(x, y_l)))]$$

This is exactly a binary classification loss where the "correct class" is always "the winner should score higher than the loser."

Level 3 (small numeric example, worked by hand as requested):

Suppose your reward model currently outputs $r(\text{explanation naming palming}) = 2.0$ and $r(\text{explanation naming misdirection-only}) = 1.0$ for the same trick clip, and a human has told you the palming explanation is the better one for this particular trick (say, one where the physical sleight-of-hand *is* the actual secret).

- Difference: $r_w - r_l = 2.0 - 1.0 = 1.0$
- Predicted probability the model already assigns to "palming explanation is preferred": $\text{sigmoid}(1.0) = 1 / (1 + e^{-1}) \approx \mathbf{0.731}$
- Loss for this single example: $-\log(0.731) \approx \mathbf{0.313}$

Now suppose instead the reward model had it backwards, $r(\text{palming}) = 1.0$, $r(\text{misdirection-only}) = 2.0$ (i.e., $r_w - r_l = 1.0 - 2.0 = -1.0$):

- Predicted probability: $\text{sigmoid}(-1.0) \approx \mathbf{0.269}$
- Loss: $-\log(0.269) \approx \mathbf{1.313}$, a much larger loss, which is exactly what you want: bigger penalty when the model disagrees with the human.

Training simply nudges the reward model's parameters, via gradient descent on this loss averaged over thousands of such pairs, until it reliably assigns higher scores to whichever response humans actually preferred.

Connection to RLHF: the reward model trained this way, $r_{\theta}(x, y)$, is exactly the $r(x, y)$ that PPO or GRPO then optimizes against in the "reinforcement learning" stage. **Connection to DPO:** as shown in Part 4, DPO derives its loss by substituting the *policy's own* implicit reward (a function of log-probabilities) into this exact same Bradley-Terry loss form, which is why DPO's loss function looks structurally identical to the reward-model loss above, just with a different definition of what "r" means.

Is a Bradley-Terry model actually necessary for your project, or might the professor mean something related instead? A few real alternatives exist and are worth knowing:

- **Generative/pairwise preference models** ("pair-pm" in the RLHFlow/RLHF-Reward-Modeling GitHub repo): instead of outputting a scalar score per response and comparing scores, the model is shown both responses together and directly outputs "A is better" or "B is better" as a classification, using the base LLM's own text-generation ability rather than a separate scoring head. This sidesteps some of Bradley-Terry's implicit assumptions (e.g., transitivity of preferences, which multiple 2025-2026 papers, like arXiv:2504.03784, note doesn't always hold for real human judgments) but is a *design choice within* the same overall preference-learning paradigm, not a replacement for it.
- **Regression/ranking losses that don't assume Bradley-Terry** (e.g., IPO, SLiC-HF - mentioned in the DPO literature search results as alternatives that use squared-error or hinge losses instead of the sigmoid form). These exist specifically because researchers have found Bradley-Terry-based losses can overfit to noisy or borderline preference pairs.
- **Best-of-N / rubric scoring without any pairwise comparison at all**: if a trick's correct explanation is programmatically checkable, you may not need *any* preference model, Bradley-Terry or otherwise (see the RLVR point in Part 3).

My assessment: Bradley-Terry is very likely exactly what the professor meant if human feedback is collected as pairwise comparisons (which is the cheapest, most common format - see Part 8), and I would not spend time hunting for an exotic alternative unless the professor specifically says your task needs to model non-transitive preferences (which would be unusual for a first project).

PART 6 - What could "better at magic tricks" actually mean?

This is the single most important open question, and I want to be direct: **I cannot determine which of these your professor meant from your notes alone.** Here is each interpretation with its I/O shape and a judgment on how "already studied" vs. "novel" it is, based on the

literature search in Part 2.

#	Interpretation	Model input	Model output	Existing work?	
A	Recognize a magic trick from video (classify: is this a trick? which category?)	Video clip	Category label / binary flag	Adjacent to general video-action recognition; no magic-specific benchmark found	Fee eas for RLI mo pre pro ste inte
B	Explain the method behind a trick	Video clip	Free-text causal explanation	No direct precedent found. Closest relative: general "explain what happened" video QA.	My gue inte tas exa kin enc jud pre out RLI me bui

C	Predict the secret state or hidden object (e.g., which hand/cup holds the coin)	Video/image, possibly with an explicit question	A specific answer (location, object identity)	Closest relative: the Zaghi-Lara et al. (2019) coin-tracking work, though that used pose estimation, not a VLM	Str can a r ver rew (gr is k from the foo ma GR frie with nee hun pre dat
---	---	---	---	--	--

D	Answer questions about a demonstrated trick (VQA-style)	Video + question	Short answer	Directly analogous to existing VideoQA benchmark formats (general, not magic-specific)	Eas buil dat qui VQ ann too con are
E	Generate correct step-by-step instructions for performing a trick	Text/video description of a trick	Ordered instruction list	Structurally identical to "procedure planning" work on COIN/ CrossTask (Part 2/7), just untried on magic content	Go war exis pro vide me dat alm dire

F	Follow a visual demonstration and reproduce the procedure (imitation)	Video demonstration	An executable action sequence (for a robot) or a matching video/ procedure	This shades into robot learning-from-demonstration, a large separate field, likely overkill for a first project given your compute	Inter give rob bac but is h Par
G	Plan a sequence of robot actions to physically perform a trick	Trick description	Robot action plan	No evidence found of this being attempted for magic specifically; general robot manipulation planning is a mature but separate field	Aln cer am a s poi req phy sim ma sta add eve else

H	Detect deception or identify misdirection in a clip	Video	Localized flag: "misdirection occurred at t=2.3s, directed attention toward X"	Psychology literature (Kuhn et al.) provides the theory; no ML implementation of this specific detector was found	High but getting truth for need when machine mis-act aim usu req ma coo or k sce foo
---	---	-------	--	--	--

I	"Mentalism" reasoning tasks (the model reasons about apparent mind-reading/prediction effects)	Description of a mentalism effect	Explanation of the psychological/procedural method	Same status as B, but narrower domain and less visual (mentalism is often more verbal/psychological than sleight-of-hand)	Plana narrow frame "ma sco spe me you
---	--	-----------------------------------	--	---	---------------------------------------

J	Learning visual procedures from demonstrations generally (magic as one example domain among several)	Any procedural video	Learned procedure representation	This is really the COIN/ CrossTask procedural-learning literature (Part 2/7) with magic as a novel, harder test domain because it deliberately includes deceptive/ irrelevant steps	Pla the pro rea pro lea bro ma we spe bec con mis ste stre mo ord coo vid
K	Evaluate whether a proposed trick will work (i.e., a "trick critic")	Description/ video of a proposed trick	Feasibility/ quality judgment	No precedent found	Inte but vali with ma exp che criti jud
L	Something else entirely	-	-	-	Car out exa the Par this que the

My honest ranking of likelihood, given your notes: B (explain the

method) and C (predict hidden state) are the two most likely candidates, because both map cleanly onto "the model gets better or worse at a checkable/preference-judgeable thing," which is the entire point of the RLHF/GRPO machinery your notes describe. C has the added advantage of being reward-function-friendly without needing human labels at all (Part 9), which would make for a faster first prototype. **I recommend proposing B and C as your two working hypotheses at the next meeting**, rather than guessing a single one and building the wrong thing for two weeks.

PART 7 - Magic tricks as a research domain: dataset design

Search results confirmed there is **no existing public dataset built specifically for magic-trick reasoning by VLMs**. You would very likely need to build a small one yourself. Below are concrete, realistically-scoped design options, mapped to the interpretations from Part 6.

Option 1: "Explain the trick" dataset (supports interpretation B)

- **Input:** a short (10-30 second) video clip of a single close-up magic trick (coin, card, or cup-and-ball tricks are easiest to source and film because they're static-camera, single-performer, and short).
- **Target output:** a human-written gold explanation of the actual mechanism (you would need either a cooperating magician, or well-documented tricks where the method is public knowledge - many classic tricks have published "exposures").
- **Labels:** for RLHF-style training, you don't strictly need "the one correct explanation" - you need *relative* judgments between two or more model-generated explanations (see Part 8), which are much cheaper to produce than a full gold-standard write-up per clip.
- **Format:** video file + a short text prompt ("Explain how this trick was likely done") + (for eval only) a gold reference explanation.
- **Number of examples needed:** realistically, **20-40 distinct trick**

clips for a first prototype (Part 12's Prototype 0-2), scaling to perhaps 150-300 for a more serious DPO/GRPO training run, which is small by ML standards but consistent with how small some published VLM-RLHF papers' custom evaluation sets are (RLHF-V's own hallucination-correction data, for comparison, was on the order of a few thousand fine-grained corrections - your first pass does not need to match that scale).

- **How humans would annotate it:** for each clip, sample 2-4 candidate explanations from the base VLM, then have a human (you, to start) rank or pairwise-compare them against a rubric derived from the Kuhn et al. misdirection taxonomy (does it correctly identify the misdirection mechanism? the physical mechanism? both?).
- **Is video necessary, or are images sufficient?** For sleight-of-hand tricks, motion is usually essential to the deception (the "moment" of the switch/palm is a specific instant), so **video is very likely necessary**, not optional, for interpretations B, C, F, H. For interpretation D (simple VQA) or E (naming steps in order), a small number of representative still frames might suffice as a cheaper starting point.
- **Evaluation metrics:** for free-text explanation quality, standard options are human/expert grading on a rubric (most reliable, most expensive), or pairwise preference win-rate of post-trained vs. pre-trained model outputs (cheaper, matches how RLHF papers usually report results), or, if you're willing to accept noise, an LLM-as-judge score.
- **Likely failure modes:** the model may hallucinate a plausible-sounding but wrong mechanism (this is the single most common VLM failure mode documented across the hallucination-focused papers in Part 2), or may over-index on superficial cues (hand near pocket -> "it went in the pocket") without actually tracking the relevant motion.

Option 2: "Predict the hidden state" dataset (supports interpretation C)

- **Input:** video up to the moment of concealment, plus a multiple-choice or short-answer question ("Which hand holds the coin

now?").

- **Target output:** the ground-truth location, obtainable if you film the tricks yourself with a second, "reveal" camera angle or by knowing the trick's mechanism in advance.
- **Labels:** this is the rare case where you can get an **objective, checkable ground truth**, meaning you could use a simple rule-based reward (exact match to the true hidden location) instead of any human preference collection at all. This is the most "GRPO-native" of all the options, closely mirroring how DeepSeekMath used exact-match correctness as its reward.
- **Number of examples needed:** because grading is automatic, you can generate more examples faster; even 30-50 filmed trick moments, each turned into multiple question variants (different phrasing, different distractor options), could give you a few hundred graded (input, question, answer) triples.
- **Video vs. images:** likely needs video up to the decision point, but the "answer" itself can be a single still frame or label, keeping the output side simple.
- **Failure modes:** the task could become trivially easy or trivially impossible depending on camera angle - if the concealment is visible on camera by accident, there's no real hidden-state reasoning happening; if it's fully hidden with no cues at all, even a perfect model cannot outperform chance, so trick selection and camera framing genuinely matter here.

Practical note on sourcing raw video

Two realistic sourcing strategies: (1) film your own trick clips (best control over camera angle, ground truth, and licensing - you own the footage outright), or (2) use existing public magic-trick or trick-exposure videos with attention to copyright and platform terms of service (usable for research/education under many institutions' fair-use guidance, but this is a legal question your professor or department, not I, should confirm before you build a dataset around scraped video - I am not a lawyer and this genuinely varies by institution and use case). **Filming your own is very likely the fastest, lowest-risk path for a first prototype**, and matches your existing hands-on technical skill set well.

PART 8 - Human feedback design

Comparison of feedback types, ordered roughly cheapest-to-most-expensive per data point:

Feedback type	What it looks like	Cost per example	Noise/ reliability	Directly Bradley-Terry compatible?
Binary success/failure	"Did the model name the correct mechanism, yes/no?"	Very cheap	Low ambiguity if you have ground truth	No (not pairwise), but trivially usable as a rule-based reward
Pairwise preference	"Which of these two explanations is better?"	Cheap - fastest to collect at volume	Some ambiguity/ inconsistency, but statistically averages out over many judgments	Yes - this is the native format Bradley-Terry expects
Ranked outputs (of 3+)	Full ranking of several candidates	Moderate	More information per item than pairwise, but harder for a human to do consistently past ~4-5 items	Can be decomposed into multiple pairwise comparisons (this is the standard trick used to train Bradley-Terry-style models on ranking data)

Scalar ratings	"Rate this explanation 1-5"	Moderate	Prone to different raters using different internal scales (calibration drift)	Only indirectly - usually needs to be converted to relative comparisons to avoid scale drift
Critiques (free text)	"This is wrong because..."	Expensive	Rich signal, but not directly a training target without more processing	No, unless converted into a correction/edit as in RLHF-V
Corrections/ edits	Human rewrites the flawed part of the output	Expensive	Very high-quality signal per example	No - typically used with dense/ segment-level DPO variants (RLHF-V's approach) rather than plain Bradley-Terry
Expert demonstrations	A human (ideally someone who actually knows magic) writes the gold explanation from scratch	Most expensive	Highest quality, but slow and requires domain expertise you may not have solo	Used for SFT/ imitation, not preference learning directly

Cheapest and most useful for a solo student researcher, realistically: pairwise preference judgments, for two reasons - they match Bradley-Terry directly (Part 5), and they are the fastest to produce in volume (a human can judge "which is better" far faster than writing a full gold explanation from scratch).

Realistic collection process for your project:

1. For each of your ~30-40 trick clips, sample 2-4 candidate explanations from the (not-yet-post-trained) base VLM at a moderate temperature (e.g., 0.7-1.0) so the samples actually differ from each other.
2. Present pairs of candidates (for 4 candidates, this is up to 6 pairs per clip, though you don't need every pair - a smaller connected subset is enough) to a human judge, with a short rubric derived from Part 7's misdirection taxonomy.
3. Record which of each pair was preferred, building a dataset of (clip, question, response_A, response_B, winner) tuples.
4. With 30 clips x ~4 pairwise judgments each, you already have roughly 120 preference pairs - enough to train a small preference/reward model as a genuine first experiment (this maps directly onto Prototype 1/2 in Part 12).

On "thousands of examples": your prompt asked how thousands of these could become training data. The honest answer for a solo/small-team student project: you likely will not personally generate thousands of high-quality pairwise judgments in a few weeks. Published VLM-RLHF work (e.g., RLHF-V) that reaches thousands of examples typically uses paid annotator pools or semi-automated labeling. A realistic path to scale, if you need it later, is a **hybrid**: collect a smaller "gold" set of human judgments yourself (100-300), then use a strong existing VLM (e.g., a larger closed or open model) as an AI judge to extend the preference dataset by another order of magnitude (this is the RLAIIF pattern from Part 4), while periodically spot-checking the AI judge's agreement with your own human judgments to make sure it isn't drifting from what you actually want.

PART 9 - Reward function design

Three concrete, distinct formulations, as requested:

Formulation 1: Rule-based / programmatic reward (best fit for interpretation C, "predict hidden state")

- **What receives the reward:** each sampled model answer to a hidden-state question.
- **How it's computed:** exact-match or near-match scoring against a known ground truth (e.g., reward = 1.0 if the model names the correct hand/location, else 0.0; optionally partial credit for naming the correct general region but wrong specific hand).
- **What makes it difficult:** you need reliable ground truth for every training example, which means either filming with a reveal angle or using tricks whose mechanism is already publicly documented; ambiguous or multi-valid-answer cases (some tricks genuinely have more than one plausible "correct" hiding spot at a given moment) need careful handling or exclusion.
- **What could be gamed:** the model could learn to always answer with the statistically most common location across your dataset rather than actually reasoning about the specific clip, if your dataset isn't balanced across hiding locations - a classic reward-hacking pattern.
- **How to validate:** hold out a test set with a different distribution of hiding locations than training, and check the model isn't just matching training-set base rates.

Formulation 2: Learned Bradley-Terry preference-based reward (best fit for interpretation B, "explain the mechanism")

- **What receives the reward:** each sampled free-text explanation.
- **How it's computed:** a small reward-model head (fine-tuned on top of a VLM or LLM) trained via the Bradley-Terry loss in Part 5 on your collected pairwise preference data, then applied at inference time to score new candidate explanations.
- **What makes it difficult:** the reward model is only as good as your (necessarily small) preference dataset; with only ~100-300 pairs,

the reward model itself may be noisy or may overfit to superficial features of your specific annotators' judgments (e.g., preferring longer answers, a well-documented reward-hacking failure mode called "length bias" that shows up repeatedly in the reward-modeling literature I searched, e.g., arXiv:2507.07375 explicitly names response length and style as common spurious correlations reward models latch onto).

- **What could be gamed:** the policy model could learn to produce longer, more confident-sounding, or more jargon-heavy explanations that *look* preferred without actually being more correct, if your human judgments happened to correlate preference with those surface features rather than accuracy.
- **How to validate:** periodically have a human directly grade a sample of post-training outputs against the *actual* correct mechanism (not just against the reward model's opinion), to check the reward model and the policy haven't co-drifted into agreeing with each other while being wrong.

Formulation 3: Hybrid reward - programmatic component + preference component

- **What receives the reward:** a weighted combination, e.g.,
$$\text{reward} = 0.5 * \text{exact_match_to_known_mechanism_keywords} + 0.5 * \text{learned_preference_score}.$$
- **How it's computed:** the programmatic half checks whether the explanation's text contains/aligns with the known correct mechanism (via keyword matching, or better, via a lightweight NLI/ entailment check against the gold mechanism description); the preference half is the Formulation 2 reward model, capturing "quality/clarity/completeness" beyond bare correctness.
- **What makes it difficult:** combining two reward signals with different scales and different noise properties requires careful weighting and normalization (this is a well-known general RLHF pain point, not specific to your project).
- **What could be gamed:** the model could learn to stuff the correct keyword into an otherwise nonsensical explanation purely to satisfy the programmatic half, without genuine reasoning - a very

common and well-documented reward-hacking pattern whenever keyword-matching is part of a reward.

- **How to validate:** spot-check high-reward outputs manually for exactly this "keyword stuffing" pattern; if you see it, tighten the programmatic component (e.g., require the keyword to appear in a coherent causal sentence, not just anywhere in the text) or down-weight it relative to the learned component.

Should the reward be learned, programmatic, human-provided, or hybrid, in general? Based on everything above: **start programmatic wherever you can** (Formulation 1, for interpretation C), because it needs no annotation infrastructure and is immune to the noisy-small-preference-dataset problems above; **add a learned preference component** (Formulation 2/3) once you need to reward free-text quality that can't be captured by simple exact-match rules. This maps directly onto the prototype ladder in Part 12.

PART 10 - VLM choice

Based on current (as of searches run today) open-source options with released weights, fine-tuning code, and RL-post-training tooling already built for them:

Model	Params (smallest useful variant)	Video support	Open weights	RL/post- training tooling available
-------	---	------------------	-----------------	--

Qwen2.5-VL-3B-Instruct	3.8B	Yes - dynamic resolution and frame-rate video understanding, per the official Qwen2.5-VL description	Yes (Apache 2.0-style Qwen license)	Yes, extensively - TRL's official <code>grpo_vlm.py</code> example, the <code>2U1/Qwen-VL-Series-Finetune</code> GitHub repo (explicitly supports LoRA/QLoRA, video training, DPO, and GRPO), verl (explicit Qwen2.5-VL support), OpenRLHF
-------------------------------	------	--	-------------------------------------	---

Qwen3-VL (various sizes, including small dense variants)	Smaller dense variants available alongside larger MoE variants	Yes, with improved long-video temporal grounding (interleaved-MRoPE, per the official repo)	Yes (Apache 2.0)	Same finetune repo (2U1) explicitly added Qwen3-VL/GRPO/ video support as of its most recent updates found in search
InternVL3 / InternVL3.5	Multiple sizes, smallest well below 10B	Yes	Yes (OpenGVLab)	Less directly confirmed RL-specific tooling in my searches compared to Qwen, though it is Hugging-Face-Transformers-compatible generally

LLaVA family (LLaVA-1.5/ LLaVA-NeXT)	7B and up	Limited/ variable across versions	Yes	This is the model used in most of the RLHF-for-VLM papers I found (LLaVA-RLHF, RLHF-V, RLAIF-V all use LLaVA variants), so there is substantial <i>prior-art</i> tooling, though possibly less actively maintained than the current Qwen ecosystem
--------------------------------------	-----------	-----------------------------------	-----	--

My recommendation, with reasoning: Qwen2.5-VL-3B-Instruct (or the equivalent small Qwen3-VL variant if you want the newest generation) is the strongest fit for a student researcher on limited hardware, for three concrete, source-backed reasons: (1) it is the model directly used in Hugging Face's own official GRPO-for-VLM tutorial and example script, meaning the "does this actually work end-to-end" question is already answered by someone else; (2) a dedicated, actively-updated open-source repo already supports LoRA, video input, DPO, *and* GRPO for this exact model family, which removes a large amount of infrastructure work you'd otherwise have to build yourself; (3) its VRAM footprint (per the third-party estimate above) is small enough to prototype on a single consumer GPU or an affordable cloud instance, which matters given your stated goal of starting small. **This is a recommendation, not a certainty** - if your professor's lab already has a specific model or compute setup, defer to that.

PART 11 - Implementation stack

Verified, not assumed, against current documentation:

- **PyTorch + Hugging Face Transformers**: the base layer everything below sits on; standard, no verification needed beyond confirming Qwen-VL models are Transformers-compatible (confirmed - `AutoModelForImageTextToText` / `AutoProcessor` pattern, seen directly in the OpenRLHF VLM docs and the Qwen-VL-Series-Finetune repo).
- **TRL (Transformer Reinforcement Learning, Hugging Face)**: confirmed to support SFT, PPO, DPO, and **GRPO with a documented Vision-Language Model training path**, including a specific `grpo_vlm.py` example script and an official blog post ("Vision Language Model Alignment in TRL") walking through multimodal GRPO with a custom reward function. This is the **most beginner-friendly** entry point of the three RL frameworks I checked, precisely because it has an official, VLM-specific worked example already.
- **PEFT / LoRA**: confirmed compatible and recommended by TRL's own VLM documentation ("Use LoRA on vision-language projection layers... VLMs are memory-intensive - start with smaller batch sizes"), and used throughout the Qwen-VL-Series-Finetune repo.
- **Accelerate / DeepSpeed**: standard multi-GPU/memory-optimization layers; relevant mainly once you move past single-GPU prototyping.
- **vLLM**: confirmed integrated into TRL's GRPO trainer for fast generation during the RL sampling step ("Most models are compatible with vLLM (server and colocate modes)"), which matters because GRPO needs to generate several completions per prompt at every training step, and naive Hugging Face generation is comparatively slow for this.
- **OpenRLHF**: confirmed to have added explicit **VLM RLHF support** (single-turn and multi-turn, with images in prompts and/or environment feedback) as of a 2026 update found in its GitHub repository/documentation, built on a Ray + vLLM distributed architecture. More production-oriented and more complex to set up than TRL; **better suited to a later, scaled-up stage of your**

project than a first prototype.

- **verl**: confirmed to explicitly support vision-language models and multi-modal RL "with Qwen2.5-VL, Kimi-VL," and to implement PPO, GRPO, GSPO, and several other algorithms. Also more infrastructure-heavy (designed around Ray-based distributed training) than TRL; a reasonable second stack to learn once you outgrow TRL's simplicity.

Do not assume a library supports a workflow just because it has "RL" in its name (per your explicit instruction) - I want to flag one concrete thing this search process ruled out: I found **no evidence** that plain `stable-baselines3` or classic robotics-RL libraries (built for continuous-control environments like MuJoCo, not for token-generation policies) are used anywhere in the current VLM-RLHF literature. Those are a different tool for a different kind of RL problem and would not be the right choice here, despite your existing robotics/RL-adjacent background making them tempting to reach for.

Minimal first-prototype architecture (informed by all of the above):

```
Qwen2.5-VL-3B-Instruct (loaded via Transformers, LoRA ada
|
v
TRL GRPOTrainer (or DPOTrainer for the cheaper first step
|
v
Reward function: Python function combining
- exact-match check against known ground truth (Formul
- a small Bradley-Terry reward head trained on your ~1
|
v
vLLM backend for fast multi-sample generation during trai
|
v
Checkpoints evaluated on held-out clips (Part 13)
```

This stack is realistic on a single consumer or single-cloud GPU for a 3B model with LoRA, based directly on the VRAM figures in Part 10 and the explicit multi-GPU-not-required framing in TRL's own VLM

documentation.

PART 12 - Start small: the prototype ladder

Your suggested six-prototype ladder was structurally reasonable; I've reordered and adjusted it based on the reward-function analysis in Part 9 (programmatic-reward tasks are strictly easier to bootstrap than preference-model tasks, so they should come first) and the algorithm comparison in Part 4 (DPO before GRPO, both before PPO).

#	Objective	Data	Model	Method	Comput
0	Baseline evaluation - how good is an off-the-shelf VLM at your chosen task, with zero training?	15-20 trick clips + questions, no labels needed yet beyond your own grading	Qwen2.5-VL-3B-Instruct, no modification	None - just inference	Minutes, any GPU or even CPU for a 3B model at small scale

1	Collect a small preference dataset	Prototype 0's outputs, sampled 2-4x per clip at temperature > 0	Same model, sampling only	Manual pairwise judgment by you, using the Part 7 rubric	Minutes to compute, hours of your own annotation time
2	Train a simple reward/preference model	Prototype 1's data	A small model (could even be a lightweight classifier on top of frozen VLM embeddings, or a small LoRA-tuned scoring head)	Bradley-Terry loss (Part 5), directly implementable in a few dozen lines of PyTorch	Single GPU, likely under an hour given the small dataset size
3	Supervised fine-tuning baseline (optional but recommended before jumping to preference optimization)	A handful of hand-written gold explanations, if you have time to write some	Base VLM + LoRA	Standard SFT	Single GPU, short run

4	Preference optimization via DPO	Prototype 1's preference pairs	Base (or Prototype-3) VLM + LoRA	DPO (no reward model needed - it uses the raw preference pairs directly)	Single GPU, no sampling loop needed, so relatively cheap and fast
5	GRPO with your Prototype-2 reward model and/or a rule-based reward	Prototype 0's clips, now used as live prompts for online sampling	Base (or Prototype-4) VLM + LoRA	GRPO via TRL's GRPOTrainer	Single GPU with vLLM for fast sampling longer run than DPO because needs repeated on-policy generation

6	Compare pretrained vs. post-trained performance rigorously	A held-out set never used in any of the above steps	All checkpoints from Prototypes 0, 3, 4, 5	Human/expert grading + automated metrics (Part 13)	Minimal additional compute, mostly your evaluation time
---	--	---	--	--	---

Why this reordering matters: your original ladder put "collect preference data" (step 1) before deciding whether the task is even well-posed (my step 0), and put GRPO/PPO (step 5) before establishing whether a much cheaper method (DPO, step 4) already gets you most of the improvement. Given your actual time constraint (days, not months), **steps 0-2 are what I'd aim to have working before your next meeting** (see Part 15), with steps 3-6 as the roadmap you present for what comes after.

PART 13 - Evaluation methodology

Held-out test set: non-negotiable from day one - carve off a fixed 20-25% of your trick clips before you do anything else, and never let the model, the reward model, or your own iterative prompt-tuning see them until final evaluation.

Expert grading: ideally by someone with real magic knowledge (even an experienced hobbyist), since you and I both lack that domain expertise; absent that, grade against published/documented trick mechanisms where available, and be explicit in your writeup about this limitation.

Pairwise preference evaluation: report win-rate of post-trained vs. pre-trained model outputs on the held-out set, judged blind (i.e., the grader doesn't know which output came from which model) to avoid your own

expectation bias.

Exact accuracy: applicable directly to interpretation C (hidden-state prediction) and D (VQA), where there's a checkable ground truth.

Rubric-based scoring: for free-text explanations (interpretation B), score along the Kuhn et al. misdirection-taxonomy dimensions separately (did it identify the *attentional* misdirection? the *physical* mechanism? both?) rather than a single vague "quality" number, so a low score is diagnosable.

Generalization checks, each testing a different failure mode:

- *Unseen tricks:* does performance hold on trick types not seen during training/preference-collection, or did the model just memorize your specific ~30 clips?
- *Unseen performers:* if you filmed all your own training clips with one person's hands, does performance transfer to a different performer's hands/style? (This tests whether the model learned something about *tricks* versus something narrow about *your specific footage*.)
- *Unseen camera angles:* does a slight camera-angle change break performance? A genuinely robust model shouldn't be this brittle.
- *Robustness to distractors:* does adding an irrelevant object or a second person into frame change the answer inappropriately?
- *Robustness to wording:* does rephrasing the same question ("what happened to the coin?" vs. "where did the coin go?") change the answer inappropriately?
- *Explanation faithfulness:* does the model's stated reasoning actually match what it would need to have "seen" to be right, or is it a plausible-sounding rationalization bolted onto a lucky guess? (This is a genuinely hard, actively-researched problem across all of LLM interpretability, not something you're expected to fully solve, but it's worth at least spot-checking qualitatively.)

Contamination and overfitting - your specific stated question, "how do we tell memorization from genuine improvement?" The most direct, practical answer given your realistic scale (dozens, not thousands, of clips): **the unseen-tricks and unseen-performers splits above are your primary tools.** If performance improves on held-out

clips featuring *tricks and performers never seen in training*, that is meaningfully harder to explain via memorization than improvement only on clips resembling training data. A second useful check: compare the *reward model's* score improvements against the *held-out human/expert grading* improvements - if the reward model's opinion goes up substantially more than actual expert-graded quality, that is a direct, quantitative signal of reward hacking/overfitting to the reward model specifically, rather than genuine task improvement. Neither check is airtight on its own (this exact problem, distinguishing genuine capability gain from narrow overfitting, is an open methodological challenge across the whole field, not something specific to your project), but together they give you an honest, defensible basis for your claims.

PART 14 - Novelty assessment (asked to be critical, so here it is)

What has already been done: the RLHF/RLAIF/DPO/GRPO methodology itself, applied to VLMs generally (LLaVA-RLHF, RLHF-V, RLAIF-V, VLM-R1), is thoroughly established. None of that methodology itself would be a novel contribution if you simply reused it unchanged.

What is incremental (useful, learnable, but not a research contribution on its own): applying an existing method (say, TRL's GRPO-for-VLM recipe) to a new domain with no methodological changes. This is exactly right for your Prototype 0-5 phase (you should absolutely do this - it's how you learn the tools), but it is not, by itself, publishable novelty.

What could be genuinely novel, ranked:

1. **A magic-trick-reasoning dataset/benchmark itself**, especially if built around interpretation C (hidden-state prediction with programmatically checkable ground truth) or B (explanation quality graded against the Kuhn et al. misdirection taxonomy). I found **no existing public benchmark of this kind** in my searches. *Novelty: high. Feasibility: high, given a filming setup and a few dozen*

tricks. Compute: low. Data requirements: moderate (tens of clips, achievable solo). Experimental clarity: high, if the task is well-posed. Likelihood of a useful result: high, even a "here's a benchmark and baseline scores" contribution is a legitimate, complete piece of research.

2. **An empirical study of whether RLHF/GRPO-style post-training transfers to a domain deliberately engineered to contain misdirection/false cues** (as opposed to the mostly-honest, mostly-cooperative visual scenes VLMs are usually trained and evaluated on). This connects directly to the "unexplored prospect" flagged in the psychology-of-magic literature (Part 2) and would be a genuine empirical contribution about VLM robustness under adversarial-by-design visual content. *Novelty: high. Feasibility: medium (needs the dataset from #1 first, plus careful evaluation design per Part 13). Compute: low-medium. Data requirements: moderate. Experimental clarity: medium (requires the memorization-vs-generalization controls from Part 13 to be done carefully). Likelihood of a useful result: medium-high.*
3. **A reward-hacking case study specific to this domain:** does a reward model trained on a small magic-trick preference dataset get gamed by the policy in domain-specific ways (e.g., confidently naming a physically plausible but wrong mechanism because it "sounds like" the kind of thing a correct answer would say)? *Novelty: medium-high (reward hacking is well-studied in general, but a domain-specific case study with a concrete, inspectable failure mode is a real contribution). Feasibility: high, this essentially falls out of doing Part 9/Part 12 carefully and honestly reporting what you find. Compute: low. Data requirements: low (you'd want it to happen, then document it). Experimental clarity: high. Likelihood of a useful result: high - even a negative or messy result here is genuinely interesting and honest.*
4. **A robot physically performing a trick (interpretation G).** *Novelty: high, if achieved. Feasibility: low for a first project - this requires a full manipulation stack, likely simulation-to-real transfer, and is a substantially larger scope than everything else discussed here. Compute: high. Data requirements: high. Experimental clarity: low (many more failure points, harder to isolate what caused a given result). Likelihood of a useful result in your available time: low. I would flag this directly to your professor as likely too ambitious for a first project, even though it's the*

most exciting-sounding option and plays directly to your robotics background - it's worth asking whether it's a long-term direction rather than a starting point.

5. **Full frontier-scale RLHF pipeline with a large, professionally-annotated preference dataset.** *Novelty: low as a method (it's the established recipe), though the resulting model/dataset could still be useful. Feasibility: very low for a solo student project - this is genuinely what industrial labs need teams and budgets for. Not recommended as a target at any point in this project's realistic scope.*

Honest bottom line: the most defensible, achievable, genuinely-novel-sounding contribution available to you is **#1 (the dataset/benchmark) combined with #3 (an honest reward-hacking case study)**, with #2 as a natural next step if #1 goes well. This gives you something concrete to show at your very next meeting (a small but real dataset and baseline numbers) while leaving room to grow toward the more ambitious framing (#2, and eventually maybe #4) over a longer timeline.

PART 15 - What to build before your next meeting

Given everything above, the single most valuable artifact you can bring is **not** a polished write-up of this document - it's evidence you've actually touched the real system. Concretely, I recommend, in priority order:

1. **A baseline evaluation script + a handful of real trick clips** (Prototype 0, Part 12). Even 5-10 self-filmed clips run through Qwen2.5-VL-3B-Instruct with a couple of candidate prompts, with your own notes on where it clearly succeeds or fails, is concrete, checkable, and directly answers "did you actually engage with the assignment."
2. **A first pass at the dataset schema** (Part 7): a short written spec (input format, output format, ground-truth format) for whichever of interpretation B or C you and the professor land on, even before you've filled it with many examples.

3. **A small hand-built preference set** (Prototype 1, Part 12): even 20-30 pairwise judgments, done by you, following a rubric you can explain and defend.
4. **This document's Part 1 (term decoding) and Part 6 (task interpretation options) as your actual talking points** - going in with "here's what I think you meant, here's my reasoning, here's what I still need you to confirm" is a far stronger opening than either guessing silently or asking an unprepared "what did you mean?"

I would **not** try to get all the way to a trained GRPO model before the next meeting unless you have unusually large amounts of free time - Part 12's own difficulty ratings show that's realistically a multi-week endeavor done properly, and a half-working, buggy GRPO run is less useful to show than a clean, honest Prototype 0 baseline plus a clear plan.

FINAL DELIVERABLE: Research proposal

Working project title

Post-Training Vision-Language Models for Reasoning Under Visual Misdirection: A Magic-Trick Case Study (working title - adjust once the professor confirms the target interpretation from Part 6)

Problem statement

Current vision-language models are evaluated almost exclusively on visual content that is honestly and cooperatively presented. It is not known how well, or how robustly, post-training methods (RLHF, DPO, GRPO) that work on ordinary visual-reasoning tasks transfer to visual content deliberately engineered to mislead a viewer through attentional or procedural misdirection - a gap explicitly noted as open in the cognitive-science-of-magic literature (Part 2) and unaddressed, as far as this search could determine, in the VLM-RLHF literature.

Hypothesis

A small, targeted post-training intervention (preference-based fine-tuning via DPO and/or GRPO, using either rule-based rewards for checkable hidden-state prediction or a Bradley-Terry preference model for explanation quality) can measurably improve a small open VLM's performance on magic-trick reasoning tasks, beyond what the base model achieves zero-shot, without simply causing the model to overfit to the small training set (verified via the held-out generalization checks in Part 13).

Proposed model

Qwen2.5-VL-3B-Instruct (or the closest available small Qwen3-VL variant), with LoRA adapters, per the reasoning in Part 10.

Dataset

A small (target: 30-50 clip) self-filmed or carefully-sourced dataset of close-up magic tricks, structured per whichever of Part 7's Option 1 (explanation quality) or Option 2 (hidden-state prediction) is confirmed with the professor, split into a training/preference-collection portion and a fixed, untouched held-out evaluation portion from the start.

Human feedback

Pairwise preference judgments (Part 8), collected by hand initially, following a rubric derived from the Kuhn et al. (2014) misdirection taxonomy, with an optional AI-judge (RLAIF-style) extension once a small human-judged "gold" set exists to validate the AI judge against.

Reward / preference model

Either a rule-based exact-match reward (if the hidden-state-prediction interpretation is chosen) or a small Bradley-Terry-loss preference model

trained on the collected pairwise data (Part 5, Part 9), with an explicit reward-hacking validation step (comparing reward-model-scored improvement against independently expert-graded improvement).

Post-training algorithm

DPO first (cheapest, no sampling loop, good for validating the pipeline quickly), then GRPO (via TRL's documented VLM-GRPO path) as the more capable second stage, with classic PPO-style RLHF deliberately deferred unless a specific need for it emerges (Part 4).

Training loop

Per Part 11's minimal architecture: Transformers + PEFT/LoRA for the base model, TRL's `DPOTrainer` then `GRPOTrainer`, vLLM for fast multi-sample generation during GRPO, all runnable on a single consumer or single cloud GPU given the VRAM estimates in Part 10.

Evaluation

Held-out clip set, blind pairwise preference win-rate, rubric-based expert/human grading along the misdirection-taxonomy dimensions, and the three generalization checks from Part 13 (unseen tricks, unseen performers, unseen camera angles) as the primary defense against the memorization-vs-genuine-improvement concern.

Baselines

(1) zero-shot base VLM, (2) SFT-only VLM (Prototype 3), (3) DPO-trained VLM, (4) GRPO-trained VLM - reported against each other so the marginal contribution of each stage is visible, not just a single before/after number.

Expected contribution

At minimum: a small, novel benchmark/dataset for magic-trick visual reasoning with documented baseline scores (Part 14, item #1) - a complete, honest, achievable contribution on its own. Potentially, if time and results allow: an empirical finding about whether/how well post-training generalizes to deliberately-misleading visual content, and/or a documented reward-hacking case study specific to this domain (Part 14, items #2-3).

Risks

Small dataset size limiting statistical confidence in any result; ambiguity in "correct" trick explanations (multiple valid causal stories for the same trick); reward hacking (Part 9); the task interpretation itself being wrong relative to what the professor actually wants (Part 6) - mitigated by confirming this explicitly at the very next meeting rather than building further on an unconfirmed guess; copyright/sourcing questions around any non-self-filmed video content (flag to the professor/department before scraping anything).

First prototype

Prototype 0 from Part 12: zero-shot baseline evaluation of Qwen2.5-VL-3B-Instruct on 5-10 self-filmed trick clips, documented with clear success/failure notes, brought to the next meeting alongside the confirmed task interpretation and dataset schema.

Questions for the professor

1. Which interpretation from Part 6 (most likely B, "explain the mechanism," or C, "predict the hidden state") is the actual target, or is it something else entirely?
2. Does "rhellf" in the original conversation mean RLHF or RLAIIF - i.e., is human labeling expected, or should I plan around an AI-judge/self-supervised approach from the start?
3. Is there an existing dataset, compute budget, or specific base model already in mind, or is building from scratch (as assumed throughout this document) correct?

4. Is video content expected to be self-filmed, or is there a sourcing/licensing pathway already approved for existing trick footage?
 5. Does the lab have a connection to the cognitive-science-of-magic/misdirection literature (Kuhn et al., and the ASU-affiliated Barnhart & Goldinger work specifically), or is that connection something I've inferred incorrectly?
 6. Is a robot-performs-the-trick end goal (interpretation G/F) part of the intended longer-term arc, or genuinely out of scope?
-

7-Day Execution Plan

Day 1: Orientation and environment setup. Read Parts 1, 4, 5, 6, and 16 (the teaching sections) of this document closely. Set up a Python environment with Transformers, PEFT, and TRL installed. Download Qwen2.5-VL-3B-Instruct and confirm you can run basic image+text inference locally or on a rented GPU.

Day 2: Baseline data. Film or source 5-10 short magic-trick clips (start with tricks whose mechanism you can verify - your own filmed sleight-of-hand, or well-documented, publicly-exposed tricks). Standardize format (resolution, length, a consistent prompt template).

Day 3: Prototype 0. Run zero-shot inference with your chosen prompt(s) on all clips for both candidate task interpretations (B and C from Part 6), in parallel, since you don't yet know which the professor wants. Write down, honestly, where the model succeeds and fails.

Day 4: Dataset schema + small preference set. Finalize the input/output schema (Part 7) for both interpretations. Sample 2-4 candidate outputs per clip from the base model. Begin pairwise preference judgments (Part 8) using the misdirection-taxonomy rubric.

Day 5: Finish preference collection + read the RL framework docs. Complete pairwise judgments (target: 60-100+ pairs across your clip set). Work through TRL's official GRPO-VLM documentation and blog post hands-on with a toy example (not your data yet) to get comfortable with the API before touching your real project.

Day 6: Toy training run. Attempt a small DPO run (Part 4, Part 12's Prototype 4) on your real preference data, even if imperfect. If time allows and the setup is stable, attempt a first GRPO run using a simple rule-based or keyword-based reward as a placeholder for the fuller Bradley-Terry reward model. Document what worked, what broke, and why, honestly.

Day 7: Consolidate and prepare for the meeting. Write a short (1-2 page) summary distilling: your term-by-term interpretation (Part 1), your two candidate task definitions with reasoning for which you lean toward (Part 6), your baseline results (Prototype 0), whatever training results you got (even partial/broken ones - "here's what happened and here's my hypothesis for why" is a strong thing to bring), and your six confirmed questions for the professor (Final Deliverable, last section). Bring this document as backup reference material, not as the thing you present verbatim.

Full Uncertainty Ledger

What we know

- Every technical term in your notes (post-training, vision-language model, GRPO, PPO, reward function, human feedback, RLHF, Bradley-Terry) is a real, current, well-documented concept, not a mistranscription of nonsense.
- GRPO was introduced by DeepSeek (DeepSeekMath, arXiv:2402.03300) and is a variant of PPO (Schulman et al., 2017, arXiv:1707.06347) that removes the need for a separately trained critic network by scoring each output against the mean/variance of a sampled group of outputs to the same prompt.
- The Bradley-Terry model (Bradley & Terry, 1952) is the standard statistical formulation underlying almost all pairwise-preference-based reward modeling in current RLHF/RLAIF work, including its role inside DPO's derivation.
- DPO (Rafailov et al., 2023, arXiv:2305.18290) and RLHF (Ouyang

et al., 2022, arXiv:2203.02155) are both real, well-documented, and frequently contrasted with each other and with GRPO in the current literature.

- There is currently no published, named subfield or benchmark specifically for "AI understanding/performing magic tricks" using VLMs and RLHF-style methods. The nearest prior work (Zaghi-Lara et al., 2019, arXiv:1908.07446) is a cognitive-science paper using pose-tracking, not a VLM-RLHF paper.
- TRL (Hugging Face), verl, and OpenRLHF all currently have documented, working support for GRPO and/or RLHF applied to vision-language models, with TRL specifically providing an official worked example using a small Qwen-VL model.
- A dedicated GitHub repository (2U1/Qwen-VL-Series-Finetune) explicitly supports LoRA, video training, DPO, and GRPO for the Qwen2/2.5/3-VL model family.

What we infer

- The professor most likely wants you to apply GRPO and/or DPO-style post-training, informed by human (or possibly AI) preference feedback modeled via Bradley-Terry, to improve a VLM's performance on some visual-reasoning task involving magic tricks, most plausibly either explaining a trick's mechanism (interpretation B) or predicting a trick's hidden state (interpretation C).
- "Magic tricks" is plausibly being used as a domain chosen specifically because it stress-tests visual reasoning under deliberate misdirection, a gap explicitly noted as unaddressed in the cognitive-science-of-magic literature - rather than magic being intrinsically the point of the research.
- "Rhelf" in your notes most likely refers to RLHF, though RLAIIF remains a real possibility given that "human feedback" and "rhelf" appear as separate bullet points in your notes.

What we do not know (confirm with the professor)

- The exact intended task interpretation (Part 6, options A-L).
- Whether "rhelf" means RLHF or RLAIIF specifically.
- Whether the professor has an existing dataset, compute

allocation, or specific base model in mind.

- Whether video is required or images suffice, and what the expected input modality actually is.
 - Whether there is a real connection to the ASU-affiliated cognitive-science-of-misdirection research (Barnhart & Goldinger, 2014) or whether that is a coincidental overlap I've inferred incorrectly.
 - Whether a robot-performs-the-trick end goal (interpretation F/G) is part of the intended scope at any stage, or genuinely out of scope for this project.
 - Any licensing/sourcing constraints on magic-trick video content the department expects you to follow.
-

References

Reinforcement learning and post-training methods

- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O. (2017). "Proximal Policy Optimization Algorithms." arXiv:1707.06347.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y.K., Wu, Y., Guo, D. (2024). "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models." arXiv:2402.03300. (Introduces GRPO.)
- DeepSeek-AI (2024). "DeepSeek-V3 Technical Report." arXiv:2412.19437. (Full GRPO objective with KL penalty.)
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C.L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). "Training Language Models to Follow Instructions with Human Feedback." arXiv:2203.02155. NeurIPS 2022. (InstructGPT / canonical RLHF recipe.)
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C.D., Finn, C. (2023). "Direct Preference Optimization: Your Language Model is Secretly a Reward Model." arXiv:2305.18290. NeurIPS 2023.
- Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., et al. (2022). "Constitutional AI: Harmlessness from AI Feedback." arXiv:2212.08073. (Anthropic; introduces RLAIFF.)

- Lee, H., et al. (2023). "RLAIF: Scaling Reinforcement Learning from Human Feedback with AI Feedback." arXiv:2309.00267.
- Bradley, R.A., Terry, M.E. (1952). "Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons." *Biometrika*, 39(3/4), 324-345.

Vision-language model post-training with human/AI feedback

- Sun, Z., et al. (2023). "Aligning Large Multimodal Models with Factually Augmented RLHF." arXiv:2309.14525. (LLaVA-RLHF.)
- Yu, T., Yao, Y., Zhang, H., He, T., Han, Y., Cui, G., Hu, J., Liu, Z., Zheng, H.-T., Sun, M., et al. (2023). "RLHF-V: Towards Trustworthy MLLMs via Behavior Alignment from Fine-grained Correctional Human Feedback." arXiv:2312.00849. CVPR 2024.
- Yu, T., et al. (2024). "RLAIF-V: Aligning MLLMs through Open-Source AI Feedback for Super GPT-4V Trustworthiness." arXiv:2405.17220.
- VLM-R1 authors (2025). "VLM-R1: A Stable and Generalizable R1-style Large Vision-Language Model." arXiv:2504.07615. Code: github.com/om-ai-lab/VLM-R1.
- Wang, Y., et al. (2024). "RL-VLM-F: Reinforcement Learning from Vision Language Foundation Model Feedback." arXiv:2402.03681.
- Rocamonde, J., Montesinos, V., Nava, E., Perez, E., Lindner, D. (2023). "Vision-Language Models are Zero-Shot Reward Models for Reinforcement Learning." arXiv:2310.12921. ICLR 2024.

Magic, misdirection, and cognitive science (not ML, but likely theoretical grounding)

- Zaghi-Lara, R., Gea, M.A., Camí, J., Martínez, L.M., Gómez-Marín, A. (2019). "Playing magic tricks to deep neural networks untangles human deception." arXiv:1908.07446.
- Kuhn, G., Caffaratti, H.A., Teshka, R., Rensink, R.A. (2014). "A psychologically-based taxonomy of misdirection." *Frontiers in Psychology*, 5:1392.
- Kuhn, G., Martinez, L.M. (2011). "Misdirection - past, present, and the future." *Frontiers in Human Neuroscience*, 5:172.
- Barnhart, A.S., Goldinger, S.D. (2014). "Blinded by magic: eye-movements reveal the misdirection of attention." *Frontiers in Psychology*. (Goldinger: Arizona State University Department of

Psychological Sciences.)

Procedural/instructional video (proxy domain for dataset structure ideas)

- Tang, Y., et al. (2019). "COIN: A Large-Scale Dataset for Comprehensive Instructional Video Analysis."
- Zhukov, D., et al. (2019). "Cross-Task Weakly Supervised Learning From Instructional Videos" (CrossTask dataset).
- Alayrac, J.-C., et al. (2016). NIV (Narrated Instructional Videos) dataset.

Frameworks and tooling (checked against current documentation, not memory)

- Hugging Face TRL documentation: GRPO Trainer, including Vision-Language Model training support - huggingface.co/docs/trl/grpo_trainer
- Hugging Face blog: "Vision Language Model Alignment in TRL" - huggingface.co/blog/trl-vlm-alignment
- verl (Volcano Engine Reinforcement Learning for LLMs), github.com/verl-project/verl - confirmed vision-language model / multimodal RL support with Qwen2.5-VL, Kimi-VL.
- OpenRLHF, github.com/openrlhf/openrlhf - confirmed VLM RLHF support (single- and multi-turn) added 2026.
- 2U1/Qwen-VL-Series-Finetune, github.com/2U1/Qwen-VL-Series-Finetune - confirmed LoRA/QLoRA, video training, DPO, and GRPO support for the Qwen-VL model family.